

HostelMint: Database & Backend Specification (Phase 1)

This specification details the database schema configurations and backend API microservices endpoints for the **HostelMint (Phase 1)** platform, strictly aligned with the official Software Requirements Document (Owner & Hostel Operations) and updated to include performance, data safety, and brute-force mitigation rules.

IMPORTANT

Official Structural Requirements (Phase 1):

- 1. **Multi-Tenancy:** Data for each hostel owner must be fully isolated. We implement the **database-per-tenant** model on PostgreSQL dynamically mapped via a Central Admin DB.
- 2. **Authentication:** JWT token-based authentication with Role-Based Access Control (RBAC). Passwords change enforced on first login for both Super Admin and Owners.
- 3. **Assets Inventory:** Simple asset list (beds, furniture, appliances) with quantity and condition.
- 4. **Receipt Generation & Reminders (Optional Addition):** Automatic PDF payment receipt generation and due-date reminders are included.
- 5. **Mobile Push Notifications:** Native support for in-app alert logs and mobile push notification registrations (FCM/APNs) is configured.
- 6. **Staff Sub-roles, Visitors, and Complaints:** Included tables for Manager/Warden permissions, Visitor registration, and Room Maintenance logs.
- 7. **Out of Scope:** Resident-facing logins, online payment gateways, and SMS/WhatsApp channels are out of scope for Phase 1.

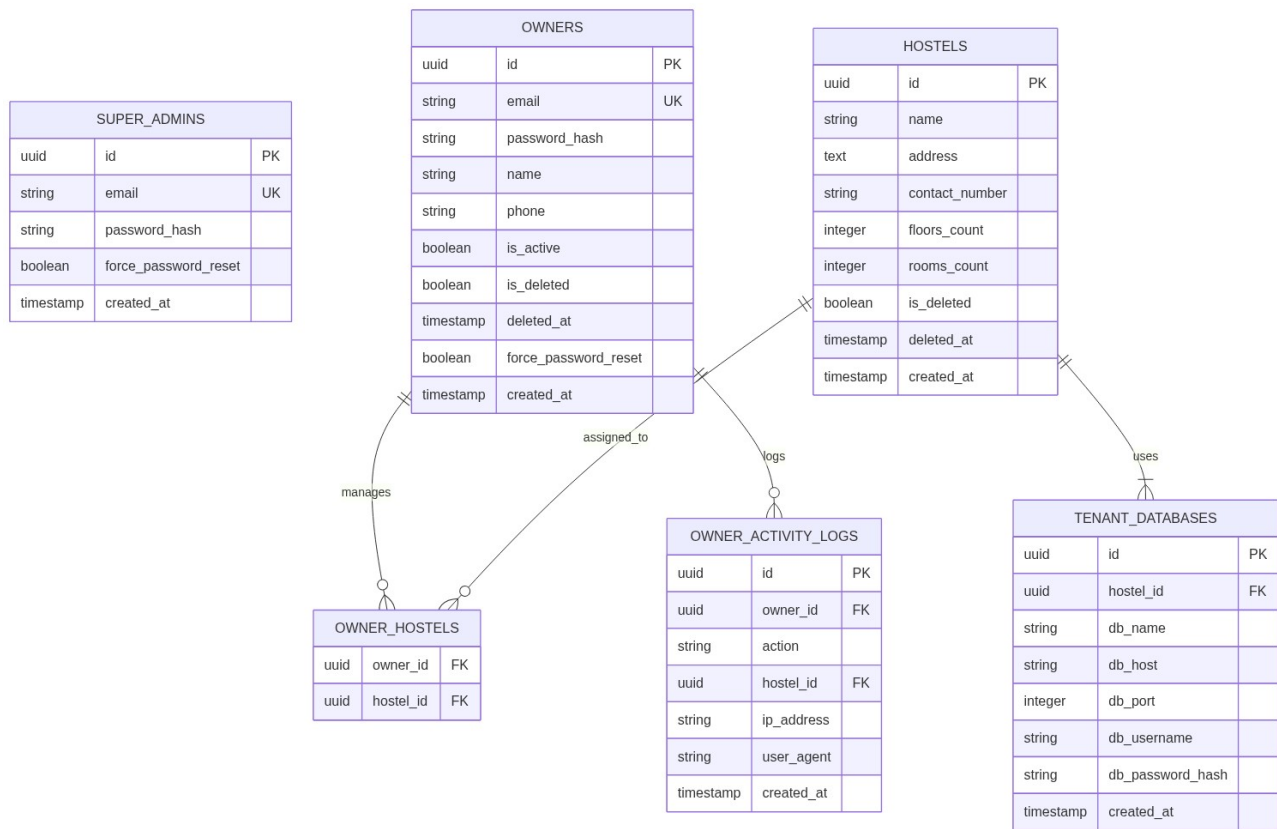
Teammate Performance & Security Additions:

- 8. **API Pagination:** All list-fetching endpoints (e.g. searching residents, fetching logs) must implement standard pagination via limit and offset parameters to prevent scale degradation.
- 9. **Soft Deletes:** To prevent accidental data loss and protect audit history, record deletion must be soft-deleted using flags (`is_deleted = TRUE` and timestamping `deleted_at`) rather than hard SQL deletions.
- 10. **Brute-force Auth Mitigation:** Rate limiting is enforced on login endpoints using Redis key counters to temporarily block accounts after 5 failed attempts in 15 minutes.
- 11. **Passwordless OTP Login & Reset:** Implemented forgot password OTP verification and passwordless login verification via Redis keys and SMTP email relays.

1. Database Configuration & Schema Design

The system implements a **multi-tenant, database-per-tenant** model using **PostgreSQL**.

- * **Central Admin DB:** Stores Super Admins, Owners, Hostel Master Records, and Tenant database routing connection details.
- * **Tenant DBs (One per Hostel):** Created dynamically. Stores isolated hostel-specific data (Hostelers, Rooms, Financials, Inventory).



1.1 Central Admin Database Schema Specs

Table: super_admins

Stores details of the singular super admin account.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
email	VARCHAR(255)	UNIQUE, NOT NULL, INDEXED	Login email.
password_hash	VARCHAR(255)	NOT NULL	Salted bcrypt hash.
force_password_reset	BOOLEAN	DEFAULT TRUE	Enforces reset on first login.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp.

Table: owners

Stores hostel owner accounts.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
email	VARCHAR(255)	UNIQUE, NOT NULL, INDEXED	Login email.
password_hash	VARCHAR(255)	NOT NULL	Salted bcrypt hash.
name	VARCHAR(100)	NOT NULL	Full name.
phone	VARCHAR(20)	NOT NULL	Contact phone number.
is_active	BOOLEAN	DEFAULT TRUE	Enables/disables dashboard access.
is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete state flag.
deleted_at	TIMESTAMP	NULLABLE	Timestamp of deletion.
force_password_reset	BOOLEAN	DEFAULT TRUE	Enforces reset on first login.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp.

Table: hostels

Stores metadata of all registered hostels.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.

name	VARCHAR(150)	NOT NULL	Name of the hostel.
address	TEXT	NOT NULL	Full physical address.
contact_number	VARCHAR(20)	NOT NULL	Hostel reception contact.
floors_count	INTEGER	NOT NULL	Total number of floors.
rooms_count	INTEGER	NOT NULL	Total number of rooms.
is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete state flag.
deleted_at	TIMESTAMP	NULLABLE	Timestamp of deletion.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp.

Table: owner_hostels

Maps Owners to Hostels (handles multi-hostel ownership).

Attribute	Datatype	Constraints	Description
owner_id	UUID	FK -> owners(id), NOT NULL	Owner ID reference.
hostel_id	UUID	FK -> hostels(id), NOT NULL	Hostel ID reference.
<i>Composite PK</i>	-	PRIMARY KEY (owner_id, hostel_id)	Prevents duplicate mappings.

Table: tenant_databases

Maps each hostel to its physical PostgreSQL database connection.

Attribute	Datatype	Constraints	Description
-----------	----------	-------------	-------------

id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
hostel_id	UUID	FK -> hostels(id), UNIQUE, NOT NULL	Associated Hostel.
db_name	VARCHAR(100)	NOT NULL	Dedicated database name.
db_host	VARCHAR(150)	NOT NULL	Database server IP/Domain.
db_port	INTEGER	NOT NULL, DEFAULT 5432	Connection port.
db_username	VARCHAR(100)	NOT NULL	Database user.
db_password_hash	VARCHAR(255)	NOT NULL	Encrypted connection password.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Connection map date.

Table: owner_activity_logs

Super admin audit trail for owner accountability.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
owner_id	UUID	FK -> owners(id), NOT NULL	Subject Owner.
action	VARCHAR(100)	NOT NULL	Action taken (e.g. ADD_HOSTELER).

hostel_id	UUID	FK -> hostels(id), NULLABLE	Hostel where action occurred.
ip_address	VARCHAR(45)	NOT NULL	IP address of request.
user_agent	VARCHAR(255)	NOT NULL	Device browser string.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Action time.

1.2 Tenant (Hostel-Specific) Database Schema Specs

Each dynamically created database contains these identical tables:

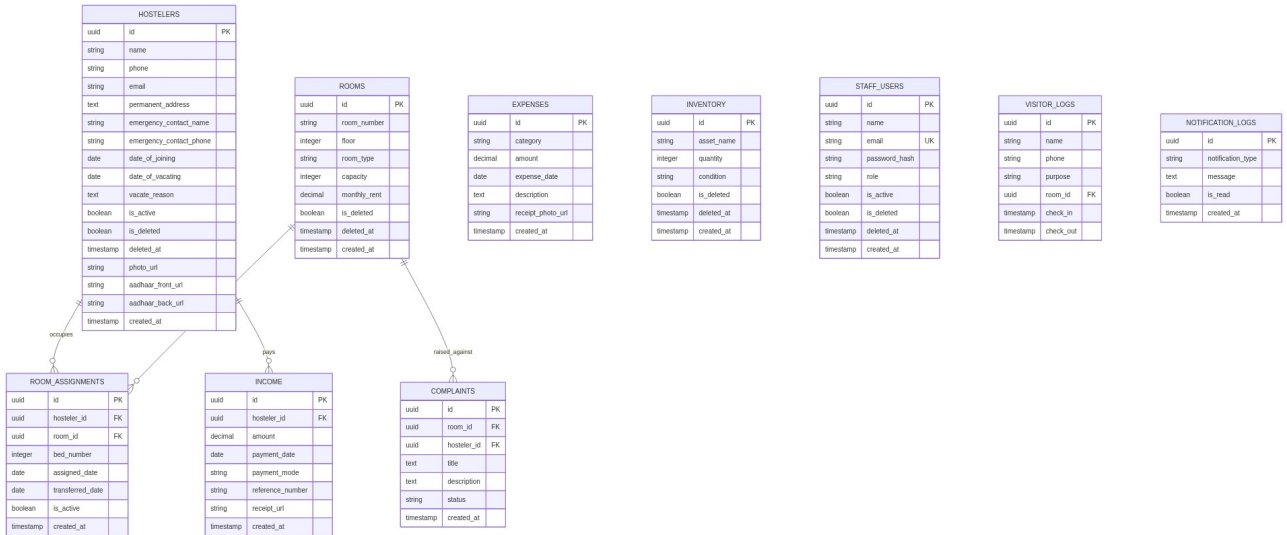


Table: hostellers

Stores profile details of active and past residents.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
name	VARCHAR(100)	NOT NULL	Full name.
phone	VARCHAR(20)	NOT NULL, INDEXED	Primary contact number.

email	VARCHAR(255)	NULLABLE	Contact email.
permanent_address	TEXT	NOT NULL	Home/Permanent address.
emergency_contact_name	VARCHAR(100)	NOT NULL	Emergency contact name.
emergency_contact_phone	VARCHAR(20)	NOT NULL	Emergency contact phone.
date_of_joining	DATE	NOT NULL	Date of check-in.
date_of_vacating	DATE	NULLABLE	Date of vacate.
vacate_reason	TEXT	NULLABLE	Reason for leaving.
is_active	BOOLEAN	DEFAULT TRUE	Active status (archived if FALSE).
is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete state flag.
deleted_at	TIMESTAMP	NULLABLE	Timestamp of deletion.
photo_url	VARCHAR(512)	NULLABLE	Secure presigned profile photo link.
aadhaar_front_url	VARCHAR(512)	NULLABLE	Secure presigned Aadhaar front.
aadhaar_back_url	VARCHAR(512)	NULLABLE	Secure presigned Aadhaar back.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Onboarding date.

Table: rooms

Hostel room inventory details.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
room_number	VARCHAR(20)	NOT NULL, UNIQUE	Room number.
floor	INTEGER	NOT NULL	Floor number (0 for Ground).
room_type	VARCHAR(20)	NOT NULL	Enum: single, double, triple, dormitory.
capacity	INTEGER	NOT NULL	Maximum beds/slots.
monthly_rent	DECIMAL(10,2)	NOT NULL	Rent cost per bed slot.
is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete state flag.
deleted_at	TIMESTAMP	NULLABLE	Timestamp of deletion.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation timestamp.

Table: room_assignments

Maps hostelers to beds and tracks room-transfer history.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
hosteler_id	UUID	FK -> hostelers(id), NOT NULL	Resident.
room_id	UUID	FK -> rooms(id), NOT NULL	Assigned Room.

bed_number	INTEGER	NOT NULL	Allocated bed number.
assigned_date	DATE	NOT NULL	Booking start date.
transferred_date	DATE	NULLABLE	End date (if transferred/vacated).
is_active	BOOLEAN	DEFAULT TRUE	Active assignment flag.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Allocation date.

Table: income

Logs rent collections.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
hosteler_id	UUID	FK -> hostelers(id), NOT NULL	Payer.
amount	DECIMAL(10,2)	NOT NULL	Amount received.
payment_date	DATE	NOT NULL	Date of transaction.
payment_mode	VARCHAR(30)	NOT NULL	Enum: cash, upi, bank_transfer, card.
reference_number	VARCHAR(100)	NULLABLE	Transaction reference (UPI/Bank ID).
receipt_url	VARCHAR(512)	NULLABLE	S3 URL of generated PDF receipt (Optional Add-on).
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Insertion date.

Table: expenses

Logs operational expenditures.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
category	VARCHAR(50)	NOT NULL	Enum: groceries, maintenance, staff_salary, electricity, water, repairs.
amount	DECIMAL(10,2)	NOT NULL	Amount spent.
expense_date	DATE	NOT NULL	Date of spending.
description	TEXT	NULLABLE	Specifics of expense.
receipt_photo_url	VARCHAR(512)	NULLABLE	S3 URL of receipt upload.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Insertion date.

Table: inventory

Tracks physical asset quantities & conditions.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
asset_name	VARCHAR(100)	NOT NULL	Asset (e.g. beds, furniture, washing_machine).

quantity	INTEGER	NOT NULL, DEFAULT 0	Count.
condition	VARCHAR(30)	NOT NULL	Enum: good, needs_repair, damaged.
is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete state flag.
deleted_at	TIMESTAMP	NULLABLE	Timestamp of deletion.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation timestamp.

Table: staff_users

Sub-roles under Owner (Manager/Warden) with limited dashboard permissions (Optional Add-on).

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
name	VARCHAR(100)	NOT NULL	Name.
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email.
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hash.
role	VARCHAR(30)	NOT NULL	Enum: manager, warden.
is_active	BOOLEAN	DEFAULT TRUE	Active flag.
is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete state flag.
deleted_at	TIMESTAMP	NULLABLE	Timestamp of deletion.

created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation timestamp.
------------	-----------	------------------------------	---------------------

Table: visitor_logs

Logs guest entries and exits (Optional Add-on).

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
name	VARCHAR(100)	NOT NULL	Guest Name.
phone	VARCHAR(20)	NOT NULL	Guest Contact.
purpose	TEXT	NOT NULL	Purpose of visit.
room_id	UUID	FK -> rooms(id), NOT NULL	Visiting Room.
check_in	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Arrival.
check_out	TIMESTAMP	NULLABLE	Departure.

Table: complaints

Tracks maintenance requests raised against specific rooms (Optional Add-on).

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
room_id	UUID	FK -> rooms(id), NOT NULL	Room ID reference.

hosteler_id	UUID	FK -> hostelers(id), NOT NULL	Resident complaining.
title	VARCHAR(150)	NOT NULL	Complaint title (e.g. Leaking tap).
description	TEXT	NOT NULL	Details of issue.
status	VARCHAR(30)	NOT NULL, DEFAULT 'open'	Enum: open, in_progress, resolved.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp.

Table: notification_logs

Tracks dispatched notifications.

Attribute	Datatype	Constraints	Description
id	UUID	PRIMARY KEY, Default: uuid_generate_v4()	Unique identifier.
notification_type	VARCHAR(50)	NOT NULL	Enum: rent_overdue, room_full, headcount.
message	TEXT	NOT NULL	Body of notification.
is_read	BOOLEAN	DEFAULT FALSE	Status.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Sent time.

1.3 Database Enum Definitions

To enforce strict constraints at the database level, the following custom PostgreSQL **ENUM types** must be declared during the migration setup.

```
-- 1. Room Type Enum (Tenant DB) CREATE TYPE room_type_enum AS ENUM ( 'single', 'double',  
'triple', 'dormitory' ); -- 2. Payment Mode Enum (Tenant DB) CREATE TYPE payment_mode_enum AS  
ENUM ( 'cash', 'upi', 'bank_transfer', 'card' ); -- 3. Expense Category Enum (Tenant DB) CREATE  
TYPE expense_category_enum AS ENUM ( 'groceries', 'maintenance', 'staff_salary', 'electricity',  
'water', 'repairs' ); -- 4. Asset Condition Enum (Tenant DB) CREATE TYPE asset_condition_enum AS  
ENUM ( 'good', 'needs_repair', 'damaged' ); -- 5. Staff Role Enum (Tenant DB) CREATE TYPE  
staff_role_enum AS ENUM ( 'manager', 'warden' ); -- 6. Complaint Status Enum (Tenant DB) CREATE  
TYPE complaint_status_enum AS ENUM ( 'open', 'in_progress', 'resolved' ); -- 7. Notification  
Type Enum (Tenant DB) CREATE TYPE notification_type_enum AS ENUM ( 'rent_overdue', 'room_full',  
'headcount' );
```

2. Microservice API Definitions

All endpoints assume JSON payloads for request/response bodies unless specified otherwise. * Headers required for Tenant-specific routes: Authorization: Bearer <JWT_TOKEN> X-Hostel-ID: <HOSTEL_UUID> (Used to locate the correct dynamic database engine)

2.0 Paginated Response Wrapping

All list-fetching API endpoints return data nested inside the following envelope to support client-side limit/offset controls:

```
{ "data": [], "pagination": { "total_records": 150, "current_page": 1, "limit": 20,  
"total_pages": 8 } }
```

2.1 Auth & Tenant Service

1. Super Admin Login

- **Endpoint:** POST /api/v1/auth/login
- **Request:** json { "email": "superadmin@hostelmint.com", "password": "SecurePassword123" }
- **Response (Success 200):** json { "token": "eyJhbGciOi...", "force_reset": false, "role": "SUPER_ADMIN" }

2. Owner Login

- **Endpoint:** POST /api/v1/auth/login
- **Request:** json { "email": "owner@email.com", "password": "TemporaryPassword" }
- **Response (Success 200):** json { "token": "eyJhbGciOi...", "force_reset": true, "role": "OWNER" }

3. Change Password (Enforce Reset)

- **Endpoint:** POST /api/v1/auth/change-password

- **Request:** json { "current_password": "TemporaryPassword", "new_password": "NewSecurePassword123" }
- **Response (Success 200):** json { "message": "Password changed successfully" }

4. Create Owner Account (Super Admin only)

- **Endpoint:** POST /api/v1/tenants/owners
- **Request:** json { "email": "john.doe@email.com", "name": "John Doe", "phone": "+919876543210" }
- **Response (Created 201):** json { "id": "e8381d64-e83c-41c3-ab0b-47e289bf4101", "email": "john.doe@email.com", "temp_password": "generated_temp_pass" }

5. Reset Owner Password / Disable Access (Super Admin only)

- **Endpoint:** POST /api/v1/tenants/owners/{id}/actions
- **Request:** json { "action": "disable" // OR "enable" or "reset_password" }
- **Response (Success 200):** json { "owner_id": "e8381d64-e83c-41c3-ab0b-47e289bf4101", "status": "disabled", "message": "Owner dashboard access has been suspended." }

6. Soft Delete Owner (Super Admin only)

- **Endpoint:** DELETE /api/v1/tenants/owners/{id}
- **Response (Success 200):** json { "owner_id": "e8381d64-e83c-41c3-ab0b-47e289bf4101", "status": "soft_deleted" }

7. Onboard New Hostel & Provision DB (Super Admin only)

- **Endpoint:** POST /api/v1/tenants/hostels
- **Request:** json { "name": "Elite Co-Living", "address": "45, Outer Ring Rd, Bangalore", "contact_number": "+918045678901", "floors_count": 4, "rooms_count": 40 }
- **Response (Created 201):** json { "hostel_id": "a9018c64-41c3-e83c-ab0b-47e289bf4055", "db_name": "hostelmint_hostel_a9018c64_db", "status": "provisioned_and_migrated" }

8. Request Forgot Password OTP

- **Endpoint:** POST /api/v1/auth/forgot-password
- **Request:** json { "email": "owner@email.com" }
- **Response (Success 200):** json { "message": "OTP has been sent to your registered email address." }

9. Verify OTP & Reset Password

- **Endpoint:** POST /api/v1/auth/reset-password
- **Request:** json { "email": "owner@email.com", "otp": "123456", "new_password": "NewSecurePassword123" }
- **Response (Success 200):** json { "message": "Password reset successfully. You can now login with your new credentials." }

10. Request Passwordless Login OTP

- **Endpoint:** POST /api/v1/auth/login-otp/request?email=owner@email.com
- **Response (Success 200):** json { "message": "Verification OTP has been sent to your email address." }

11. Authenticate with Passwordless Login OTP

- **Endpoint:** POST /api/v1/auth/login-otp/verify
- **Request:** json { "email": "owner@email.com", "otp": "123456" }
- **Response (Success 200):** json { "token": "eyJhbGciOi...", "force_reset": false, "role": "OWNER" }

2.2 Hostel & Room Service

1. Search & Filter Hostellers (Paginated)

- **Endpoint:** GET /api/v1/hostellers
- **Query Parameters:**
 - page (optional integer, default: 1)
 - limit (optional integer, default: 20)
 - search (optional string for Name/Phone search)
 - room_number (optional string)
 - is_active (optional boolean, default: true)
 - joining_date (optional string, format: YYYY-MM-DD)
- **Response (Success 200):** json { "data": [{ "id": "c1c38381-e83c-41c3-ab0b-47e289bf4801", "name": "Rahul Sharma", "phone": "+919988776655", "room_number": "101", "date_of_joining": "2025-01-10", "is_active": true }], "pagination": { "total_records": 1, "current_page": 1, "limit": 20, "total_pages": 1 } }

2. Add New Hosteler

- **Endpoint:** POST /api/v1/hostellers

- **Request:** json { "name": "Amit Kumar", "phone": "+918877665544", "email": "amit.k@gmail.com", "permanent_address": "Flat 3A, Green Valley, Patna", "emergency_contact_name": "Suresh Kumar", "emergency_contact_phone": "+918877665500", "date_of_joining": "2025-05-01", "photo_key": "uploads/temp_profile_1.jpg", "aadhaar_front_key": "uploads/temp_aadhaar_f.jpg", "aadhaar_back_key": "uploads/temp_aadhaar_b.jpg" }
- **Response (Created 201):** json { "id": "d05f3d81-e83c-41c3-ab0b-47e289bf4222", "status": "active" }

3. Edit Hosteler & Mark Vacated

- **Endpoint:** PUT /api/v1/hostelers/{id}
- **Request:** json { "is_active": false, "date_of_vacating": "2025-07-01", "vacate_reason": "Completed studies" }
- **Response (Success 200):** json { "hosteler_id": "d05f3d81-e83c-41c3-ab0b-47e289bf4222", "status": "archived_vacated" }

4. Soft Delete Hosteler

- **Endpoint:** DELETE /api/v1/hostelers/{id}
- **Response (Success 200):** json { "hosteler_id": "d05f3d81-e83c-41c3-ab0b-47e289bf4222", "status": "soft_deleted" }

5. Add Room

- **Endpoint:** POST /api/v1/rooms
- **Request:** json { "room_number": "101", "floor": 1, "room_type": "triple", "capacity": 3, "monthly_rent": 8500.00 }
- **Response (Created 201):** json { "room_id": "f8381d64-e83c-41c3-ab0b-47e289bf4333", "status": "created" }

6. Assign Bed Space

- **Endpoint:** POST /api/v1/rooms/assign
- **Request:** json { "hosteler_id": "d05f3d81-e83c-41c3-ab0b-47e289bf4222", "room_id": "f8381d64-e83c-41c3-ab0b-47e289bf4333", "bed_number": 2, "assigned_date": "2025-05-01" }
- **Response (Success 200):** json { "assignment_id": "b649ea15-9846-4b99-3e57-29815953e036", "status": "allocated" }

7. Re-assign / Transfer Room

- **Endpoint:** POST /api/v1/rooms/transfer
- **Request:** json { "hosteler_id": "d05f3d81-e83c-41c3-ab0b-47e289bf4222", "new_room_id": "a9018c64-e83c-41c3-ab0b-47e289bf4999", "new_bed_number": 1 }

- **Response (Success 200):** `json { "message": "Transfer completed successfully",
"new_assignment_id": "c025bbc9-cd73-fa17-a82d-04bdf8e9adc1" }`
-

3. Important Implementation Instructions

3.1 Pydantic Validation & Error Handling

All APIs must parse and validate inputs using **Pydantic v2** models. Unhandled exceptions must be caught by custom FastAPI error handlers to return clean HTTP responses.

3.2 Dynamic Migration Runner

Because we have isolated PostgreSQL databases for each hostel, standard static database migrations will not work. When the Super Admin creates a new hostel, the backend must programmatically invoke **Alembic** (or strict SQL schema runner scripts) to apply schemas:

```
def run_tenant_migrations(db_connection_url: str): # Initializes types, schemas, and indices dynamically ...
```

3.3 Asynchronous Task Execution (Celery + Redis)

To avoid blocking request threads, long-running operations are pushed to a Redis task queue and executed by Celery workers:

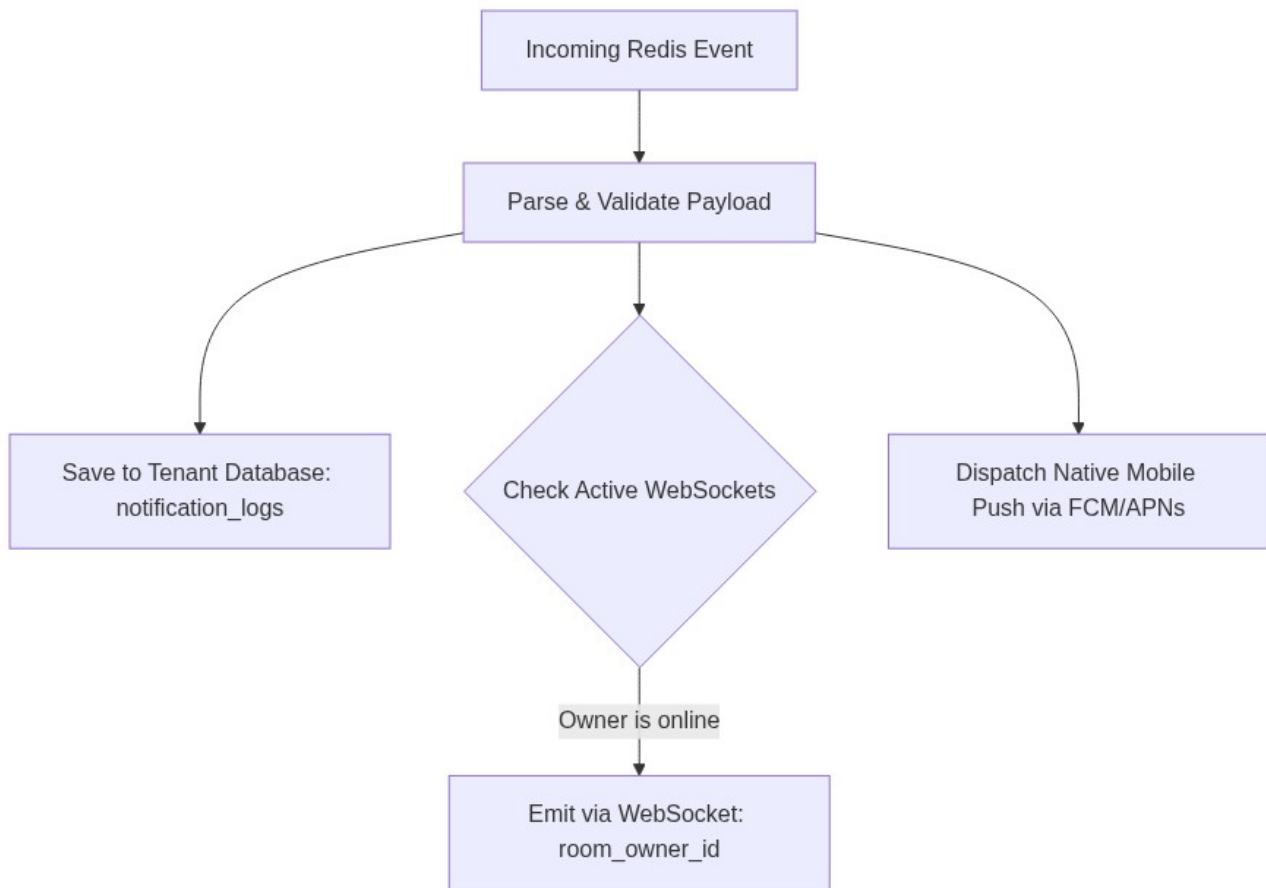
1. **Monthly Headcount Capture:** A cron job running at midnight on the 1st of every month to loop through active databases and capture occupancy snapshot logs.
2. **Notification Dispatch (APNs / FCM):** Handled by queues mapping to Firebase Cloud Messaging for native mobile push alerts when a room is filled or rent is overdue.
3. **Optional Receipt Compilation:** Compilation of receipt PDF files and S3 uploads is handled in background queues.

3.4 API Validation & Business Constraints

- **Password Rules:** Minimum 8 characters, must contain at least one uppercase letter, one lowercase letter, one digit, and one special character.
 - **Over-allocation Prevention:** Before allocating a hosteler to a room, verify `active_assignments < capacity`. If equal, return HTTP 400 Bad Request with code `ROOM_CAPACITY_EXCEEDED`.
 - **Positive Financials:** Transaction amounts must be strictly positive (`amount > 0.00`).
 - **Future Block:** Transaction and usage logs cannot be dated in the future.
-

3.5 Notification Service Event Processing Loop

The Notification Service acts as a decoupled subscriber to the Redis message bus. It processes events and routes them across In-App Logs and APNs/FCM Push:



3.6 Secure Document Storage & Retrieval (Aadhaar & Receipt Vault)

To ensure absolute privacy of sensitive personal records (Aadhaar card scans) and financial documents, backend developers must implement a **presigned URL pattern** using S3-compatible object storage (AWS S3 or MinIO):

1. Bucket Access Policy

- The S3/MinIO bucket must be configured as **strictly private**.
- Public read, write, or list operations must be blocked by default at the bucket level.

2. Presigned Upload (POST/PUT) Protocol

- Frontend clients must never upload files directly through the backend API servers to prevent thread clogging and network bottlenecks.
- **Upload Flow:**
 - The client calls `POST /api/v1/hostelers/presigned-upload` requesting an upload slot.
 - The backend generates a temporary presigned POST policy dictionary using `boto3` in Python:

```
python response = s3_client.generate_presigned_post( Bucket='hostelmint-secure-vault', Key=f"uploads/{hostel_id}/aadhaar/{uuid.uuid4()}.jpg", Fields={"Content-Type": "image/jpeg"}, Conditions=[{"Content-Type": "image/jpeg"}], ExpiresIn=300 # Valid for 5 minutes )
```

- The backend returns this response payload to the client. The client performs a direct HTTPS POST request to upload the file binary directly from the device to S3.
- Only the unique S3 Object Key string (e.g. uploads/<hostel_id>/aadhaar/<uuid>.jpg) is saved in the hosteler database fields (aadhaar_front_url Or aadhaar_back_url).

3. Secure Retrieval (GET) Protocol

- The backend must never store or return public static URLs to files.
- **Retrieval Flow:**
- The client requests hosteler details (GET /api/v1/hostelers/{id}).
- The backend fetches the raw S3 Key from the database.
- The backend generates a short-lived presigned GET URL (valid for **120 seconds**):

```
python
secure_url = s3_client.generate_presigned_url( ClientMethod='get_object', Params={'Bucket':
'hostelmint-secure-vault', 'Key': db_stored_s3_key}, ExpiresIn=120 # URL expires in 2 minutes
)
```
- The backend returns this URL. The frontend uses it to download and display the image securely.

3.7 Tenant Connection & Request Lifecycle Middleware

Backend developers must implement a request middleware to handle dynamic routing across isolated PostgreSQL database instances:

1. **Extraction:** For every incoming request to a tenant route, the middleware reads the x-Hostel-ID header or extracts the hostel_id claim from the verified JWT token.
2. **Lookup:** The middleware queries the Central Admin DB to retrieve the specific connection string credentials (host, port, username, password, database name) associated with that Hostel ID.
3. **Session Management:**
4. The backend maintains an in-memory dictionary cache of SQLAlchemy Engine instances keyed by hostel_id.
5. If a connection pool (Engine) does not exist in the cache, the middleware instantiates it and appends it.
6. It spawns a new database session (db_session) and binds it to the active request thread using Python's contextvars.
7. **Clean-up:** Once the controller yields the response, the middleware intercepts the lifecycle to commit or rollback the transaction, close the session, and return the connection to the PgBouncer pool.

3.8 Asynchronous Audit Logging Logic

To ensure absolute accountability of owner actions (Super Admin audit trail), backend developers must record every state-modifying event:

- **Trigger:** Any successful request to endpoints under `/api/v1/hostelers/*`, `/api/v1/rooms/*`, and `/api/v1/finance/*` must log an audit record.
- **Flow:**
 - The API controller successfully executes the core database transactions.
 - Instead of blocking the HTTP thread, the backend pushes an audit log job payload to the Redis queue.
 - A background Celery worker consumes the task and performs a quick insert directly into the Central Admin database `owner_activity_logs` table: `sql INSERT INTO owner_activity_logs (owner_id, action, hostel_id, ip_address, user_agent) VALUES (:owner_id, :action_type, :hostel_id, :client_ip, :user_agent);`

3.9 Authentication Rate Limiting (Brute-Force Prevention)

To protect login endpoints (POST `/api/v1/auth/login` and passwordless options) from dictionary or brute-force requests, developers must implement a rate-limiting filter using **Redis**:

1. **Key Tracking:** Upon every failed authentication attempt, the backend increments a Redis string key keyed by the user's email address: `failed_attempts:{email}`.
2. **Lockout Trigger:** - If the Redis counter value reaches **5 attempts**, the backend blocks further requests for that email, returning HTTP 429 Too Many Requests with a custom error message. - The Redis block key is configured with a **15-minute TTL (Time-To-Live)**.
3. **Reset:** Upon a successful login attempt, the backend deletes the `failed_attempts:{email}` key to restore the clean slate.

3.10 Soft Delete Query Conventions

To prevent physical SQL DELETE executions from removing historical resident, room, and inventory records, all entities include `is_deleted` and `deleted_at` fields:

- * **Query Filter:** Backend developers must override ORM default select queries (e.g. using SQLAlchemy's `with_loader_criteria` or custom query filters) to automatically append `WHERE is_deleted = FALSE` to database reads.
- * **Deletion Executions:** Running a deletion API (DELETE `/api/v1/.../{id}`) must run an update statement: `sql UPDATE <table_name> SET is_deleted = TRUE, deleted_at = NOW() WHERE id = :id;`
- * **Recovery API:** Add support for a restoration route POST `/api/v1/.../{id}/restore` to undo accidental updates by reverting `is_deleted` to FALSE and nullifying the timestamp.

4. Microservice Folder Structure & Architecture Layout

Each microservice must be built in Python (FastAPI) following a modular, testable directory structure:

```
backend/<service_folder>/ app/ main.py # FastAPI application entry point & CORS config core/
config.py # Environment variables (Docker-compose variables mapping) security.py # JWT Token
decoding/encryption operations middlewares/ tenant_routing.py # DB-connection injection or auth
hooks models/ db_models.py # SQLAlchemy Declarative Models (Schema definitions) schemas/
api_schemas.py # Pydantic v2 schemas for request bodies & response models routers/ api_routes.py
# FastAPI Route definitions (APIRouter endpoints) controllers/ business_logic.py # Pure business
logic (DB sessions execution, checks) alembic/ # Database migrations directory (only in services
modifying DBs) Dockerfile # Container instructions requirements.txt # Python dependencies
```

Role of Layers:

1. **Routers (Interface Layer):**
 2. Handle path declarations, query parameters parsing, HTTP verbs (GET, POST, PUT), and OpenAPI status tags.
 3. Inject dependencies (e.g., Auth verification, active Database sessions).
 4. **Rule:** Keep routers thin. They should do nothing except validate schemas and forward requests to the controller.
 5. **Controllers (Business Logic Layer):**
 6. House all the actual logic (e.g. bed occupancy checks, calculation algorithms, database commits, S3 presigned key triggers).
 7. Must operate inside database transactions, calling commit/rollback.
 8. **Middlewares (Interception Layer):**
 9. Run hooks on incoming request headers or outgoing responses (e.g., database connection switching based on X-Hostel-ID, logging audit parameters).
-

5. Backend Implementation Task List (Task Division)

This checklist organizes developer duties by implementation phase to facilitate division of labor:

Phase 1: Shared Core Infrastructure

- [x] **Task 1.1:** Set up root directory with `docker-compose.yml` defining PostgreSQL, Redis, MinIO, and Nginx.
- [x] **Task 1.2:** Create a mock S3 MinIO storage container configuration and buckets setup script.
- [x] **Task 1.3:** Configure the Nginx API Gateway (`nginx.conf`) routing requests to `/api/v1/auth/*` and `/api/v1/hostel_service/*`.
- [x] **Task 1.4:** Define the master `.env.example` file.
- [x] **Task 1.5:** Set up base microservice boilerplate `requirements.txt` and `Dockerfiles`.

Phase 2: Central Database & Auth Service (`auth_service/`)

- [x] **Task 2.1:** Implement Central Admin DB schemas (SuperAdmins, Owners, Hostels, TenantDatabases) supporting soft delete flags.
- [x] **Task 2.2:** Implement login routes for Super Admin and Owners (JWT token generation).
- [x] **Task 2.3:** Implement first-time password change routing constraint middleware.
- [x] **Task 2.4:** Implement Super Admin Owner onboarding (`POST /api/v1/tenants/owners`) and access suspension toggle endpoints.
- [x] **Task 2.5:** Implement Redis failed-attempts lockout counter rate limiter middleware on login routes.
- [x] **Task 2.6:** Write script to seed the default Super Admin credentials on deployment.

Phase 3: Tenant Connection Middleware & Alembic Runner

- [x] **Task 3.1:** Create dynamic database connection routing middleware (engine cache directory dictionary).
- [x] **Task 3.2:** Write programmatic Alembic migration trigger script to initialize identical tenant schemas ON CREATE DATABASE.
- [x] **Task 3.3:** Set up Tenant Alembic configuration environment.

Phase 4: Hostel & Room Service (`hostel_service/`)

- [x] **Task 4.1:** Implement Tenant DB schemas (Hostelers, Rooms, RoomAssignments) supporting soft delete columns.
- [x] **Task 4.2:** Build Room creation endpoint (`POST /api/v1/rooms`) supporting 2, 3, or 4 beds capacity constraints.
- [x] **Task 4.3:** Build Hosteler registration endpoint (`POST /api/v1/hostelers`) tracking Aadhaar upload keys.
- [x] **Task 4.4:** Implement Bed Allocation checks (`POST /api/v1/rooms/assign`) to prevent over-allocation.
- [x] **Task 4.5:** Implement SQL Transaction block for Room Transfers (`POST /api/v1/rooms/transfer`) logging historical occupancy movements.
- [x] **Task 4.6:** Integrate limit/offset query pagination for Hosteler listing.
- [x] **Task 4.7:** Build soft deletion and restoration endpoints for Hostelers and Rooms.

Phase 5: Finance & Asset Inventory Service (`finance_service/`)

- [] **Task 5.1:** Implement Tenant DB schemas (Income, Expenses, Inventory) supporting soft deletes.
- [] **Task 5.2:** Build Income logging endpoint (`POST /api/v1/finance/income`) verifying positive payment amount values.
- [] **Task 5.3:** Build Expense logging endpoint (`POST /api/v1/finance/expenses`) checking receipt uploads and category Enums.
- [] **Task 5.4:** Build Asset inventory listing endpoint (`GET /api/v1/finance/inventory`) with pagination support.
- [] **Task 5.5:** Build custom date-range summary report endpoint (Income vs. Expenses vs. Net Profit).
- [] **Task 5.6:** Build soft deletion and restoration endpoints for Inventory.

Phase 6: Notification & Event Service (`notification_service/`)

- [] **Task 6.1:** Implement Tenant DB schema (NotificationLogs).
- [] **Task 6.2:** Establish Redis Event Bus listener loop to subscribe to published events (e.g. `rent_overdue`, `room_full`).

- [] **Task 6.3:** Implement WebSocket Room Manager in FastAPI mapping active owners by ID.
- [] **Task 6.4:** Integrate FCM/APNs push notification triggers for mobile alerts.
- [] **Task 6.5:** Integrate SMTP email sender tasks in Celery background queues.
- [] **Task 6.6:** Add paginated listing endpoints for Notification Logs.

Phase 7: Storage Service (`storage_service/`)

- [] **Task 7.1:** Implement presigned POST upload URL generator endpoint for Aadhaar images.
- [] **Task 7.2:** Implement presigned GET download URL generator endpoint with 120-second validity constraints.
- [] **Task 7.3:** Configure local MinIO client boto3 connections for local docker development environment testing.

Phase 8: Containerization & Integration Testing

- [] **Task 8.1:** Write Dockerfiles for each of the 5 microservices.
- [] **Task 8.2:** Integrate services in `docker-compose.yml` mapping environment secrets.
- [] **Task 8.3:** Set up Swagger/OpenAPI aggregation at the API Gateway level.
- [] **Task 8.4:** Perform end-to-end integration tests.

6. Environment Configuration & Security Protocols

6.1 Environment Variable Schema (`.env.example`)

Developers must configure a `.env` file in the root of each microservice container. Below is the master schema containing all required keys:

```
# --- Service General Config --- APP_ENV=production # development, staging, production
API_VERSION=v1 SECRET_KEY=generate_a_secure_random_hex_string_32_bytes # --- Database
Configurations ---
CENTRAL_DATABASE_URL=postgresql://central_admin:password@central-db:5432/hostelmint_admin_db
PG_BOUNCER_POOL_MODE=transaction MAX_TENANT_CONNECTIONS=100 # --- Caching & Event Bus ---
REDIS_URL=redis://redis-broker:6379/0 # --- Secure File Vault (S3/MinIO) ---
S3_ENDPOINT_URL=https://s3.amazonaws.com # Mock MinIO URL for local dev
S3_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
S3_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
S3_BUCKET_NAME=hostelmint-secure-vault S3_REGION_NAME=ap-south-1 # --- Real-Time Push
Notification Credentials --- FCM_SERVER_KEY=firebase_cloud_messaging_secret_server_key
APNS_KEY_ID=apple_push_notification_key_id APNS_TEAM_ID=apple_developer_team_id # ---
Transactional Mail SMTP Relay --- SMTP_HOST=smtp.brevo.com SMTP_PORT=587
SMTP_USERNAME=relay@hostelmint.com SMTP_PASSWORD=brevo_smtp_secret_password
SMTP_FROM_EMAIL=alerts@hostelmint.com
```

6.2 API Versioning Policy

- All microservice routes must be prefixed with `/api/v1/` (e.g. `/api/v1/auth/login`, `/api/v1/hostelers`).
- This enables forward-compatibility. When Phase 2 (resident-facing application) is deployed, we can introduce `/api/v2/` endpoints if schemas or route parameters change, without interrupting or breaking active Phase 1 React Native mobile clients.

6.3 CORS & Security Header Rules

- **FastAPI CORS Middleware Configuration:**

The API Gateway and individual services must enforce strict CORS validation rules: ``python from fastapi.middleware.cors import CORSMiddleware

```
app.add_middleware( CORSMiddleware, allow_origins=[ "https://admin.hostelmint.com", # Production
Web Admin "http://localhost:3000", # Local Web Admin dev port ], allow_credentials=True,
allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"], allow_headers=["Authorization",
"Content-Type", "X-Hostel-ID"], ) `` - Mobile clients (React Native) bypass standard browser
CORS checks, but requests must still carry validated JWT authorizations. * **HTTP Security
Headers**: - The API Gateway must inject standard security headers on all responses:
-X-Frame-Options: DENY(prevents clickjacking). -X-Content-Type-Options: nosniff(prevents
MIME-type sniffing). -Content-Security-Policy: default-src 'self' (strict CSP).
```